

# EXPERIMENT 05 – HTML WEB TABLE EXTRACTOR

## 1. Module Name and Experiment Title

**Module Name:** Module 1 – Automation with Selenium

**Experiment Number:** Experiment 05

**Experiment Title:** HTML Web Table Extractor

**Programming Language:** Python

**Automation Tool:** Selenium WebDriver

---

## 2. Problem Statement

Web applications often display information in HTML tables containing multiple rows and columns. Manually extracting this information can be time-consuming and inefficient.

The objective of this experiment is to automate the extraction of data from an HTML web table using Selenium WebDriver. The automation script should locate the table, identify its headers, extract the data from each row, and display the extracted information in the terminal.

To make the extraction process visually observable, the script should highlight each row while processing it. It should also capture screenshots and close the browser automatically after completing the extraction.

---

## 3. Objective

The main objectives of this experiment are:

1. To automate the extraction of data from an HTML web table using Selenium WebDriver.
  2. To locate a specific web table using its HTML ID.
  3. To identify and extract the table headers.
  4. To extract data from each row and its corresponding cells.
  5. To display the extracted table data in the terminal.
  6. To highlight each row during extraction to make the process visually observable.
  7. To capture screenshots of the web page and extraction process.
  8. To verify successful extraction and close the browser automatically.
- 

## 4. Tools, Software and Concepts Used

## Tools and Software:

- **Python:** Programming language used to write the automation script.
- **Selenium WebDriver:** Used to automate browser interactions and extract table data.
- **Google Chrome:** Browser used to execute the automation.
- **Visual Studio Code:** Code editor used to write and execute the Python script.

## Concepts Used:

- Selenium WebDriver initialization
- Browser automation
- HTML table identification
- CSS Selectors
- WebDriverWait
- HTML table headers and rows
- Data extraction using `find_elements()`
- JavaScript execution using `execute_script()`
- Row highlighting
- Screenshot capture
- Assertions and exception handling

## Website Used:

<https://rahulshettyacademy.com/AutomationPractice/>

**HTML Table Used:** Web Table Example

**Table ID:** product

---

## 5. Source Code / Implementation Steps

### Python Source Code

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
import time
import os

# Create screenshots folder
os.makedirs("screenshots", exist_ok=True)

# Initialize WebDriver
driver = webdriver.Chrome()
wait = WebDriverWait(driver, 20)

try:
    # Step 1: Open and maximize browser
    driver.maximize_window()
    driver.get("https://rahulshettyacademy.com/AutomationPractice/")
    time.sleep(2)

    # Step 2: Locate the HTML web table
```

```

table = wait.until(
    EC.presence_of_element_located((By.ID, "product"))
)

# Scroll to the table
driver.execute_script(
    "arguments[0].scrollIntoView({block: 'center'});", table
)
time.sleep(2)

# Step 3: Capture initial table screenshot
driver.save_screenshot("screenshots/01_initial_web_table.png")
print("Screenshot 1: Initial table saved.")

# Step 4: Extract table headers
headers = table.find_elements(By.CSS_SELECTOR, "th")
header_names = [header.text.strip() for header in headers]

print("\nTable Headers:")
print(header_names)

# Step 5: Locate table rows
rows = table.find_elements(By.CSS_SELECTOR, "tbody tr")
extracted_data = []

print("\nStarting row-by-row extraction...")

# Step 6: Extract data from each row
for index, row in enumerate(rows[1:], start=1):

    # Scroll to the current row
    driver.execute_script(
        "arguments[0].scrollIntoView({block: 'center'});", row
    )

    # Highlight the current row
    driver.execute_script("""
        arguments[0].style.backgroundColor = 'yellow';
        arguments[0].style.outline = '3px solid red';
        """, row)

    # Extract cell data
    cells = row.find_elements(By.CSS_SELECTOR, "td")
    row_data = [cell.text.strip() for cell in cells]

    if row_data:
        extracted_data.append(row_data)
        print(f"Row {index}: {row_data}")

    # Pause to make row highlighting visible
    time.sleep(1)

    # Remove highlighting
    driver.execute_script("""
        arguments[0].style.backgroundColor = '';
        arguments[0].style.outline = '';
        """, row)

# Step 7: Verify extracted data
assert header_names, "Table headers not found!"
assert extracted_data, "No table records found!"

```

```

# Step 8: Display extraction summary
print("\nTotal records extracted:", len(extracted_data))
print("HTML Web Table extraction successful!")

# Step 9: Capture final screenshot
driver.save_screenshot("screenshots/02_extracted_web_table.png")
print("Screenshot 2: Extracted table saved.")

time.sleep(2)

finally:
    # Step 10: Close the browser automatically
    driver.quit()
    print("Browser closed successfully.")

```

## Implementation Steps

### Step 1: Initialize the WebDriver

Import the required Selenium modules and initialize Chrome WebDriver. Maximize the browser window.

### Step 2: Open the Website

Navigate to the Automation Practice website and wait for the page to load.

### Step 3: Locate the Web Table

Locate the Web Table Example using its HTML ID, `product`. Scroll the page until the table is visible.

### Step 4: Capture the Initial Screenshot

Capture a screenshot showing the web table before the extraction process.

### Step 5: Extract the Table Headers

Locate the table header elements using the CSS selector `th` and extract their text.

### Step 6: Extract Table Rows

Locate the table rows using the CSS selector `tbody tr`. Iterate through the data rows and extract the text from each cell.

### Step 7: Highlight Rows During Extraction

Use JavaScript to highlight each row with a yellow background and red outline. A one-second delay makes the extraction process visible in the browser.

### Step 8: Verify the Extracted Data

Use assertions to verify that the table headers and records have been successfully extracted.

### Step 9: Display the Results

Print the table headers, extracted records, total number of records, and success message in the terminal.

### Step 10: Capture the Final Screenshot and Close the Browser

Capture the final screenshot and automatically close the browser using `driver.quit()`.

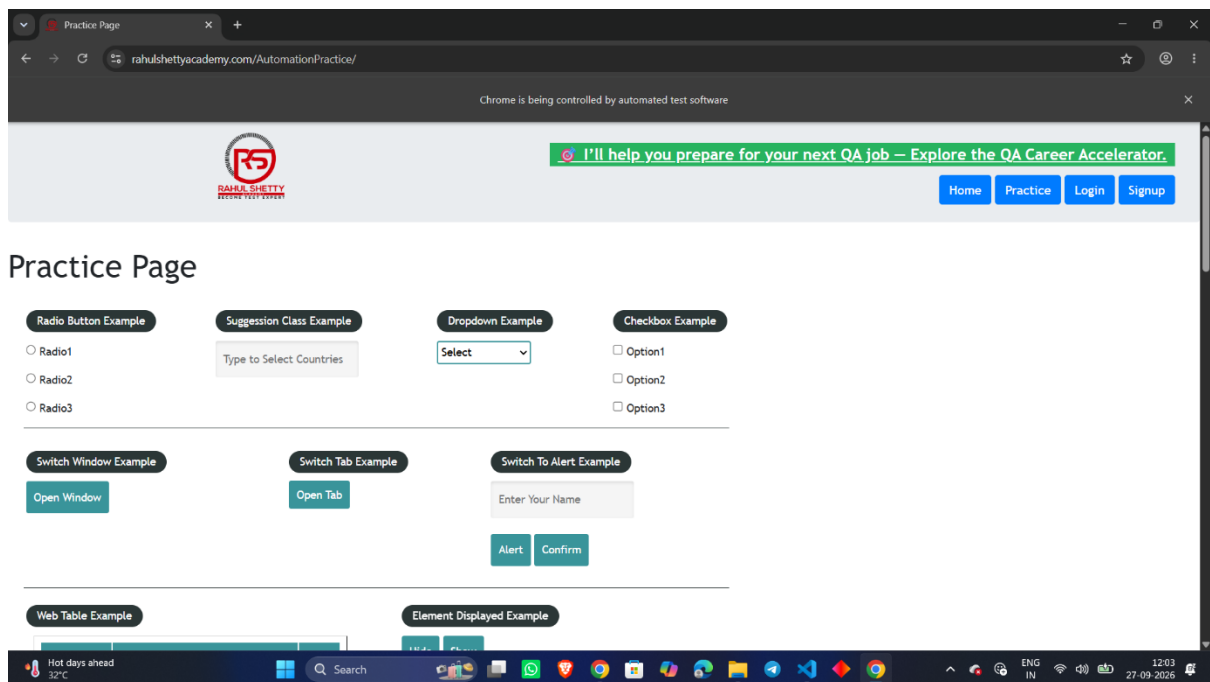
## 6. Output with Clearly Labelled Screenshots

The automation script was executed successfully using Selenium WebDriver and Google Chrome. The script located the HTML table, extracted its headers and records, and displayed the results in the VS Code terminal.

The following four screenshots demonstrate the different stages of the experiment.

## Screenshot 1: Initial Web Page

**Description:** The initial state of the Automation Practice website before the table extraction process begins.



## Screenshot 2: HTML Web Table Displayed

**Description:** The Web Table Example displaying the Instructor, Course, and Price columns. This is the table used for data extraction.

Web Table Example

| Instructor   | Course                                                           | Price |
|--------------|------------------------------------------------------------------|-------|
| Rahul Shetty | Selenium Webdriver with Java Basics + Advanced + Interview Guide | 30    |
| Rahul Shetty | Learn SQL in Practical + Database Testing from Scratch           | 25    |
| Rahul Shetty | Appium (Selenium) - Mobile Automation Testing from Scratch       | 30    |
| Rahul Shetty | WebSecurity Testing for Beginners- QA knowledge to next level    | 20    |
| Rahul Shetty | Learn JMeter from Scratch - (Performance + Load) Testing Tool    | 25    |
| Rahul Shetty | WebServices / REST API Testing with SoapUI                       | 35    |
| Rahul Shetty | QA Expert Course :Software Testing + Bugzilla + SQL + Agile      | 25    |
| Rahul Shetty | Master Selenium Automation in simple Python Language             | 25    |
| Rahul Shetty | Advanced Selenium Framework Pageobject, TestNG, Maven, Jenkins,C | 20    |
| Rahul Shetty | Write effective QA Resume that will turn to interview call       | 0     |

Element Displayed Example

Hide Show

Hide/Show Example

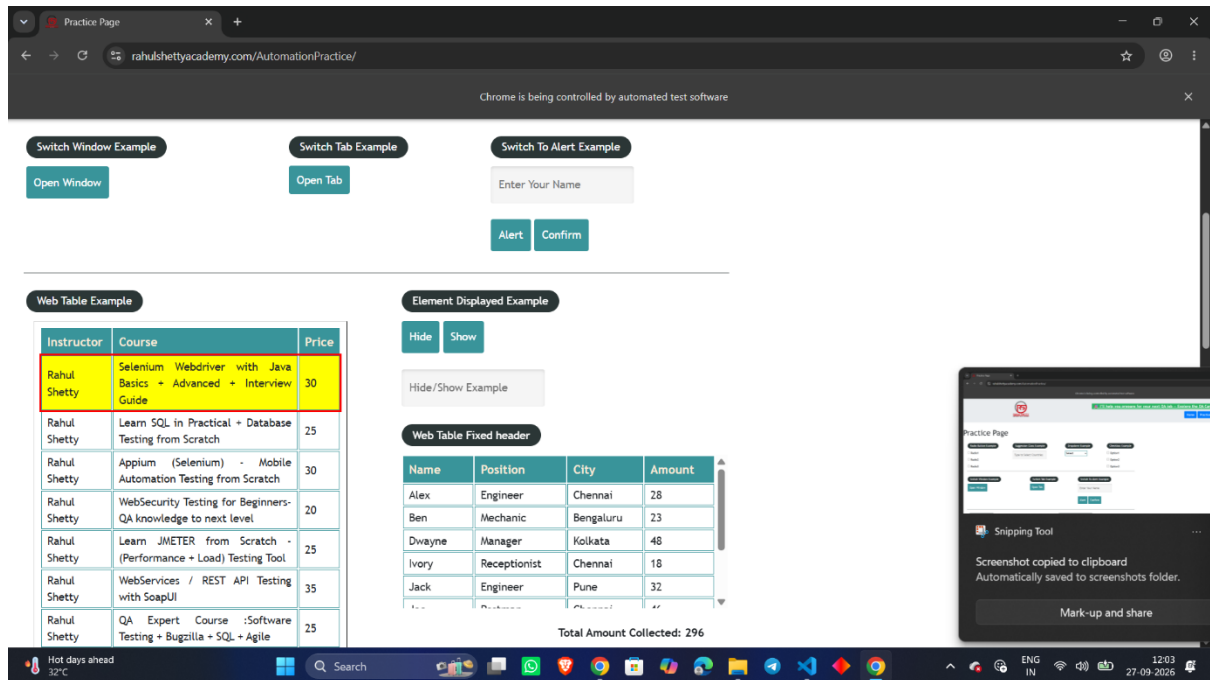
Web Table Fixed header

| Name   | Position     | City      | Amount |
|--------|--------------|-----------|--------|
| Alex   | Engineer     | Chennai   | 28     |
| Ben    | Mechanic     | Bengaluru | 23     |
| Dwayne | Manager      | Kolkata   | 48     |
| Ivory  | Receptionist | Chennai   | 18     |
| Jack   | Engineer     | Pune      | 32     |

Total Amount Collected: 296

## Screenshot 3: Row-by-Row Extraction

**Description:** A table row highlighted with a yellow background and red outline while Selenium extracts its data.



## Screenshot 4: Successful Execution and Console Output

**Description:** The VS Code terminal displaying the extracted table records, the total number of records, and the successful execution message.

### Console Output:

Screenshot 1: Initial table saved.

Table Headers:

```
['Instructor', 'Course', 'Price']
```

Starting row-by-row extraction...

```
Row 1: ['Rahul Shetty', 'Selenium Webdriver with Java Basics + Advanced + Interview Guide', '30']
```

```
Row 2: ['Rahul Shetty', 'Learn SQL in Practical + Database Testing from Scratch', '25']
```

```
Row 3: ['Rahul Shetty', 'Appium (Selenium) - Mobile Automation Testing from Scratch', '30']
```

```
Row 4: ['Rahul Shetty', 'WebSecurity Testing for Beginners-QA knowledge to next level', '20']
```

```
Row 5: ['Rahul Shetty', 'Learn JMeter from Scratch - (Performance + Load) Testing Tool', '25']
```

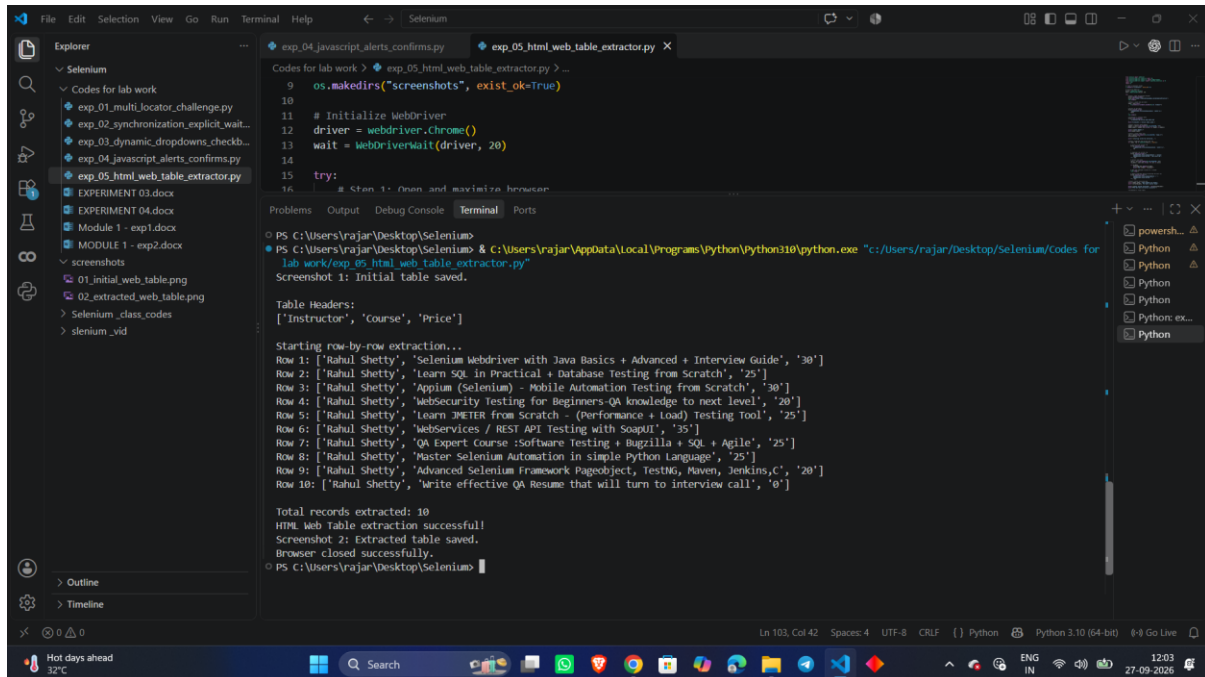
```
Row 6: ['Rahul Shetty', 'Webservices / REST API Testing with SoapUI', '35']
```

```
Row 7: ['Rahul Shetty', 'QA Expert Course :Software Testing + Bugzilla + SQL + Agile', '25']
```

```
Row 8: ['Rahul Shetty', 'Master Selenium Automation in simple Python Language', '25']
```

Row 9: ['Rahul Shetty', 'Advanced Selenium Framework Pageobject, TestNG, Maven, Jenkins,C', '20']  
Row 10: ['Rahul Shetty', 'Write effective QA Resume that will turn to interview call', '0']

Total records extracted: 10  
HTML Web Table extraction successful!  
Screenshot 2: Extracted table saved.  
Browser closed successfully.



```
9 os.makedirs("screenshots", exist_ok=True)
10
11 # Initialize WebDriver
12 driver = webdriver.Chrome()
13 wait = WebDriverWait(driver, 20)
14
15 try:
16     # Step 1: Open and maximize browser
17     driver.get("https://rahulshettyacademy.com/AutomationPractice")
18     driver.maximize_window()
19
20     # Step 2: Extract table data
21     table = driver.find_element_by_xpath("//table").get_attribute("innerHTML")
22     table_data = table.split("\n")
23     headers = table_data[0].split("<th>")
24     records = []
25     for row in table_data[1:]:
26         record = row.split("<td>")
27         records.append(record)
28
29     # Step 3: Save data to CSV
30     with open("extracted_data.csv", "w") as f:
31         f.write(",".join(headers) + "\n")
32         for record in records:
33             f.write(",".join(record) + "\n")
34
35     # Step 4: Take screenshot
36     driver.save_screenshot("screenshot_2.png")
37
38     # Step 5: Close browser
39     driver.quit()
40
41 except Exception as e:
42     print(f"Error: {e}")
43
44 if __name__ == "__main__":
45     main()
```

Problems Output Debug Console Terminal Ports

PS C:\Users\rajar\Desktop\Selenium> . C:\Users\rajar\AppData\Local\Programs\Python\Python310\python.exe "c:/Users/rajar/Desktop/Selenium/codes for lab work/exp\_05\_html\_web\_table\_extractor.py"

Screenshot 1: Initial table saved.

Table Headers:  
['Instructor', 'Course', 'Price']

Starting row-by-row extraction...

Row 1: ['Rahul Shetty', 'Selenium WebDriver with Java Basics + Advanced + Interview Guide', '30']  
Row 2: ['Rahul Shetty', 'Learn SQL in Practical + Database Testing from Scratch', '25']  
Row 3: ['Rahul Shetty', 'Applium (Selenium) - Mobile Automation Testing from Scratch', '30']  
Row 4: ['Rahul Shetty', 'WebSecurity Testing for Beginners-QA knowledge to next level', '20']  
Row 5: ['Rahul Shetty', 'Learn JMeter from Scratch - (Performance + Load Testing Tool)', '25']  
Row 6: ['Rahul Shetty', 'WebServices / REST API Testing with SoapUI', '35']  
Row 7: ['Rahul Shetty', 'QA Expert Course :Software Testing + Bugzilla + SQL + Agile', '25']  
Row 8: ['Rahul Shetty', 'Master Selenium Automation in simple Python Language', '25']  
Row 9: ['Rahul Shetty', 'Advanced Selenium Framework Pageobject, TestNG, Maven, Jenkins,C', '20']  
Row 10: ['Rahul Shetty', 'Write effective QA Resume that will turn to interview call', '0']

Total records extracted: 10  
HTML Web Table extraction successful!  
Screenshot 2: Extracted table saved.  
Browser closed successfully.

PS C:\Users\rajar\Desktop\Selenium>

## 7. Result, Observation and Conclusion

### Result

The HTML Web Table Extractor experiment was successfully completed using Python and Selenium WebDriver.

The automation script successfully:

- Opened the Automation Practice website.
- Located the Web Table Example using its HTML ID.
- Extracted the table headers: Instructor, Course, and Price.
- Extracted all 10 data records from the table.
- Highlighted each row during the extraction process.
- Displayed the extracted data in the VS Code terminal.
- Captured screenshots of the experiment.
- Closed the browser automatically after execution.

### Observation

The experiment demonstrated that Selenium WebDriver can locate HTML table elements and extract data from their rows and columns. JavaScript was used to highlight each row, making the extraction process visually observable. The extracted data was displayed in the terminal, and the assertions verified that the headers and records were present.

## **Conclusion**

The experiment successfully demonstrated automated HTML web table extraction using Selenium WebDriver and Python. It provided practical experience in locating web elements, extracting table data, using CSS selectors, executing JavaScript, and capturing screenshots. The experiment also demonstrated how automation can reduce the need for manual data extraction from web pages.